

Construction Element Detection for Site Progress Verification

A cloud-only YOLOv8 pipeline for automated progress evidence

Progress evidence is captured, then lost

- **Latency.** Two to seven days between the photograph and the progress record. Decisions get made on stale data.
- **Inconsistency.** Two engineers describe the same floor differently. No controlled vocabulary links what is visible to a structured record.
- **Loss of information.** The photograph is archived. What is *in* it — steelbar placed, blockwork started, conduit run — is never captured as data, so it cannot enter a CDE or be audited.

In the Gulf, progress reporting on large programmes is still photograph-plus-narrative, with a two-to-seven-day lag between capture and record. Government BIM mandates and ISO 19650 information-delivery requirements are raising the bar on structured, auditable progress evidence. A detection record — image, time, zone, class, confidence — is structured information that can enter a CDE. A JPEG in a folder is not.

Five classes, one detector, zero local installs

Detect construction elements and plant in site photographs, then use presence as a progress and site-condition signal.

Class	Signal it carries
steelbar	Reinforcement activity — structural progress
brick	Masonry and blockwork — envelope progress
pvcpipe	Services installation — MEP first fix
scaffold	Temporary works standing — zone not released
excavator	Plant present — earthworks active

Pipeline

- Roboflow — fork, extend, annotate
- Colab — YOLOv8, 30 epochs, T4
- Validate — P / R / mAP, per class
- Evidence — validation + unseen images
- Handover — repo, governance, licence

A public base, re-split and extended

Forked **seungyeon/construction-site-km7bh** (700 images, CC BY 4.0) into our own Roboflow workspace, re-split it 80/20, and added our own images including deliberate boundary cases — the things that look like a class and are not.

Boundary case	Looks like	Actually is	Rule it tests
_____	steelbar	Loose scaffold tube on the ground	Belongs to NEITHER class
_____	scaffold	Ladder, handrail, hoarding frame	Erected access, not any frame
_____	brick	Stone cladding, paving block	Material versus form
_____	pvcpipe	Metal conduit under dust	Material when colour fails
_____	excavator	Wheeled loader, dozer	Machine type, not yellow plant

Why boundary cases matter. A class is defined by its edge, not its centre. Without written rules and images that test them, annotation drifts between people and the model learns the drift.

Validation performance

0.950

Precision

0.884

Recall

0.943

mAP@50

0.809

mAP@50-95

Class	P	R	mAP@50	mAP@50-95
brick	0.984	1.000	0.995	0.932
excavator	0.969	1.000	0.995	0.883
pvcpipe	0.986	0.930	0.964	0.835
scaffold	0.955	0.875	0.979	0.858
steelbar	0.855	0.612	0.781	0.535

Success criteria. S1 overall mAP@50 ≥ 0.50 — _____. S2 steelbar recall ≥ 0.50 — _____. S3 brick recall ≥ 0.40 — _____. Targets were set before the run and are reported whether met or missed.

Three things the numbers say

- 1 All three success criteria were met, and that is the least interesting result.** mAP@50 reached 0.943 against a 0.50 target. Clearing a bar that far is itself a signal that the measurement is wrong, not that the model is excellent.
- 2 The split leaks.** Consecutively numbered frames sit on both sides of it (steelBar_5397k in train, 5398k in validation); mAP@50 was already 0.677 after one epoch; two classes reach recall 1.000. The headline number is not a generalisation estimate.
- 3 steelbar is weakest in both architectures tried** — 0.781 here, 0.539 in an independent YOLOv11n cross-check — despite having the most training instances. Shape and our group-boxing rule explain it; instance count does not. Per-class ranking survives the leak; absolute values do not.

The two errors are not symmetrical

False negative

Severity: High

Steelbar is placed, the model misses it. No record is created. Nobody investigates, because nobody knows there is anything to investigate. **The error is silent.**

False positive

Severity: Moderate

The model reports steelbar that is not there. The next site walk contradicts it. **The error is loud and self-correcting** — but repeated, it produces alert fatigue.

One model, two thresholds

Progress logging runs at a **low** confidence threshold, favouring recall because a miss is silent. Site-condition alerts run at a **high** one, favouring precision because false alarms train people to ignore the system. A single threshold would be wrong for one of them.

Three false positives and three false negatives are documented with filenames and hypotheses in *docs/error_analysis.md*, each tied to a dataset gap rather than to model capacity — and each paired with a concrete improvement and the metric it should move.

When not to use this

- The reported metrics are inflated by train/validation leakage from near-duplicate frames. They measure memorisation as much as detection. Re-splitting by group is improvement D1.
- No safety function. The model detects no people, no PPE and no hazards. Treating it as a safety system would reduce site safety, not improve it.
- Presence only. It cannot measure quantity, spacing, cover or conformance, and its detection counts must never enter a payment application.
- Validated scope is daytime site photography at working range. Night, dust, rain, drone and enclosed-space imagery are untested.

Rights

CODE

MIT

DATASET

CC BY 4.0, not owned

WEIGHTS

AGPL-3.0 inherited

PEOPLE DETECTED

None, by design

Commercial deployment of YOLOv8-derived weights requires AGPL-3.0 compliance or an Ultralytics licence. This is the most commonly missed obligation in YOLO projects.